

PROJECT DOCUMENTATION REPORT

SHOP MANAGEMENT & E-COMMERCE SYSTEM

A Scalable, Cloud-Connected Enterprise Store, Inventory & Order Management Platform Built with Django & Neon Serverless PostgreSQL

Project Title:	Shop Management & E-Commerce System
Lead Developer:	Md Jamshed Alam • ID: 2023100010017
Project Contributors:	• Sk. Ashiqur Rahman Akash (ID: 2023100010022) • Mirza Safunur Rahman (ID: 2023100010251)
Technology Stack:	Python 3.14, Django 6.1.1, Neon PostgreSQL, DRF, JWT
Currency Format:	Bangladeshi Taka (৳ BDT)
Document Purpose:	Academic Project Submission & Faculty Review

Submitted for: Academic & Technical Evaluation

Date: September 2026

Table of Contents

Section	Topic & Discussion Details	Page
1.0	Executive Summary & Project Objectives	2
2.0	Technology Stack & System Architecture	2
3.0	Database Design & Relational Data Models	3
4.0	Core Features & Functional Modules	4
5.0	Cloud Database Integration (Neon PostgreSQL)	5
6.0	Inventory & Order State-Machine Engine	5
7.0	Security, Authentication & Role Access	6
8.0	Project Setup, Execution & Testing Guide	6
9.0	Conclusion & Future Enhancements	7

1.0 Executive Summary & Project Objectives

The **Shop Management System** is a full-featured web-based e-commerce platform and inventory control system. It is designed to cater to modern retail businesses requiring an intuitive customer storefront alongside an enterprise-grade administrative back-office.

In traditional retail systems, inventory discrepancies and disconnected storefronts cause order delays, stockouts, and administrative overhead. This system solves these issues through real-time stock synchronization, automated transaction logging, order lifecycle management, and a seamless dual-tier user experience (public storefront for shoppers and secured portal for administrative staff).

Key Project Objectives:

- **Public Accessibility:** Provide an open, responsive storefront where customers can browse catalog items, search products, and view item specifications without mandatory login barriers.
- **Robust Cloud Data Layer:** Utilize Neon Serverless PostgreSQL for resilient, cloud-hosted relational storage with connection pooling.
- **Automated Inventory Control:** Prevent human errors by automatically deducting stock when orders dispatch and restoring stock upon authorized returns.
- **Secured Staff Portal:** Offer JWT-authenticated administrative tools with role-based access for catalog, inventory, and order operations.
- **Localization:** Native presentation of Bangladeshi Taka (৳) currency symbols and clean, modern user interfaces.

2.0 Technology Stack & System Architecture

The system employs a multi-tiered architecture separating the presentation layer, the application business logic, and the cloud data persistence layer.

Tier	Technology	Version	Description & Function
Backend Core	Python / Django	3.14 / 6.1.1	Core web framework managing ORM, URL routing, business services, and views.
API Layer	Django REST Framework	3.18.1	Provides RESTful endpoints for frontend clients and future mobile applications.

Tier	Technology	Version	Description & Function
Authentication	SimpleJWT & Djoser	5.5.1 / 2.3.4	Stateless JSON Web Token authentication with cookie and header validation.
Database Engine	Neon PostgreSQL	18.0 Cloud	High-performance serverless PostgreSQL connected via <code>psycopg3</code> 3.3.6 .
Frontend UI	HTML5, Vanilla CSS, JS	ES6+	Lightweight, fast client interface utilizing Bootstrap 5.3 and Bootstrap Icons.
Configuration	python-dotenv & dj-database-url	Latest	Secure management of cloud credentials and dynamic database URL routing.



3.0 Database Design & Relational Data Models

The database schema is organized in the `store` and `auth_app` applications to maintain strong normalization, entity integrity, and transactional consistency.

3.1 Core Entity Models & Schema Details

Entity Model	Primary Attributes	Relationships & Constraints
User (<code>auth_app</code>)	<code>email</code> , <code>full_name</code> , <code>phone</code> , <code>is_staff</code> , <code>is_active</code>	Custom User Model using email as unique login identifier. Role-based flags.
ProductType	<code>title</code> , <code>slug</code> , <code>description</code> , <code>is_active</code>	Parent classification for products (e.g., Clothing, Electronics, General).
Collection	<code>title</code> , <code>slug</code> , <code>image</code> , <code>image_url</code> , <code>is_active</code>	Marketing collections (e.g., Summer Collection, Featured, New Arrivals).
Product	<code>title</code> , <code>slug</code> , <code>sku</code> , <code>main_price</code> , <code>old_price</code> , <code>new_price</code> , <code>stock</code> , <code>image</code>	Central catalog model. FK to <code>ProductType</code> (PROTECT) and <code>Collection</code> (SET_NULL). Unique SKU constraint.
ProductChoice & Value	<code>title</code> , <code>value</code> , <code>slug</code>	Dynamic product options (e.g., Size: S, M, L, XL; Color: Black, White, Red).
ProductVariant	<code>sku</code> , <code>price_override</code> , <code>stock</code> , <code>is_active</code>	M2M relationship with <code>ProductChoiceValue</code> . Represents sellable SKU variations.
Order	<code>order_number</code> , <code>customer_name</code> , <code>customer_phone</code> , <code>subtotal</code> , <code>total</code> , <code>status</code> , <code>payment_status</code>	Order lifecycle root. Stores delivery address, discount, tax, and order totals.
OrderItem	<code>quantity</code> , <code>unit_price</code> , <code>subtotal</code> , <code>returned_quantity</code>	Line items linking to <code>Product</code> and optional <code>ProductVariant</code> .
InventoryTransaction	<code>transaction_type</code> , <code>quantity</code> , <code>stock_before</code> , <code>stock_after</code> , <code>reason</code>	Immutable audit log capturing all stock deductions, restocks, and manual adjustments.

Entity Model	Primary Attributes	Relationships & Constraints
OrderStatusLog	<code>old_status</code> , <code>new_status</code> , <code>changed_by</code> , <code>note</code> , <code>created_at</code>	Immutable audit log recording every order state transition with user attribution.

Data Integrity Highlight: The `Product` model implements `on_delete=models.PROTECT` for product types to prevent accidental cascade deletion of active inventory. All financial calculations utilize Python's `Decimal` data type to eliminate floating-point rounding inaccuracies.

4.0 Core Features & Functional Modules

4.1 Public Storefront (Customer Facing)

The public storefront at `/` is specifically designed for accessibility without requiring user registration or login:

- **Interactive Hero Banner:** Eye-catching introductory banner with store highlights, customer assurances (Authentic Items, Fast Dispatch, 24/7 Care), and call-to-action buttons.
- **Live Search & Instant Filtering:** Real-time JavaScript search filtering products by title, SKU, or collection without triggering full page reloads.
- **Dynamic Category Filter Pills:** Instant category switching (All Products, Electronics, Clothing, etc.) powered by database records.
- **Localized Currency Display:** All active product cards, discounts, and modal details format pricing using the Bangladeshi Taka symbol (`৳`).
- **Quick View Modal:** Comprehensive product detail dialog revealing high-resolution images, SKU identification, stock availability count, and long-form descriptions.

4.2 Staff Administration & Dashboard Portal

Administrative operations are managed through two specialized interfaces:

- **Django Advanced Admin (`/admin/`):**
 - Custom branding with dedicated management titles.
 - Visual status badges for stock health (In Stock, Low Stock, Out of Stock).
 - Inline management for product gallery images and choice variants.
 - Batch administrative actions (e.g., Dispatch Orders, Process Returns, Toggle Active status).
- **Dedicated Staff Dashboard (`/dashboard/` & `/login/`):**
 - Secured by JWT authentication and custom permission mixins.
 - **One-Click Demo Credentials:** The login page features a dedicated Demo Credentials card (`admin@gmail.com` / `admin`) with an instant "Quick Fill" button for academic reviewers and testing personnel.
 - Direct links between the storefront and staff dashboard for smooth navigation.

4.3 Order Processing & State-Machine Workflow

Orders transition through a rigorously validated state machine to ensure business rules are enforced:

Pending → Confirmed → Processing → Dispatched → Delivered
(Secondary branches: Cancelled , Return Requested , Returned , Partially Returned)

5.0 Cloud Database Integration (Neon PostgreSQL)

The system has transitioned from local SQLite storage to enterprise **Neon Serverless PostgreSQL**. This provides remote accessibility, high concurrency, and production reliability.

5.1 Configuration Architecture

The database connection is managed securely via environment variables and Django's `dj-database-url` parser:

```
# .env Configuration
DATABASE_URL=postgresql://neondb_owner:***@ep-young-moon-b3c0j242-pooler.c-4.ap-southeast-1.aws.

# config/settings.py
DATABASES = {
    "default": dj_database_url.config(
        default=os.getenv("DATABASE_URL"),
        conn_max_age=600,
        conn_health_checks=True,
    )
}
```

Cloud Benefits Achieved:

- **Remote Cloud Accessibility:** The database is hosted in AWS Asia-Pacific (ap-southeast-1), allowing deployments on render/Vercel/VPS without local SQLite locking issues.
- **Connection Pooling:** Utilizing Neon's PgBouncer pooler ensures resilient handling of high-concurrency connections.
- **SSL Encryption:** All data in transit between Django and Neon is encrypted using mandatory SSL/TLS mode (`sslmode=require`).

6.0 Inventory & Order Service Layer

To prevent business logic leakage into views or admin controllers, the project implements a dedicated **Service Layer** (`store/services.py`):

Service Class	Method	Business Operation & Automation
InventoryService	dispatch_order()	Deducts physical stock from <code>Product</code> and <code>ProductVariant</code> when an order changes status to <code>dispatched</code> . Automatically logs an <code>order_dispatch</code> transaction in <code>InventoryTransaction</code> .
InventoryService	return_order_items()	Validates return eligibility, restores inventory counts, and creates an audit record tagged as <code>return</code> with user attribution.
StatusService	change_status()	Enforces allowed status transitions, verifies state rules, and writes an entry into <code>OrderStatusLog</code> .
OrderService	calculate_totals()	Computes item subtotal, applies dynamic discounts, calculates tax and shipping charges, ensuring consistent totals.

7.0 Security, Authentication & Role Access

Security is implemented across multiple defensive layers:

- **JWT Token Management:** SimpleJWT generates dual token pairs (Access Tokens with short expiry and Refresh Tokens for prolonged sessions).
- **Cookie-Based Secure Storage:** Access tokens are transported with `HttpOnly`, `SameSite=Lax`, and cryptographic signing to protect against XSS and CSRF attacks.
- **Custom JWTAuthentication & Mixin:** The `JWTRequiredMixin` inspects cookies and authorization headers, verifying user credentials and staff clearance before granting access to dashboard views.
- **Password Encryption:** Uses PBKDF2 with SHA256 algorithm and salt hashing, conforming to OWASP password storage recommendations.

8.0 Project Setup, Execution & Testing Guide

Follow these steps to initialize and run the application locally or in a presentation environment:

8.1 Prerequisites & Virtual Environment

```
# 1. Open Terminal in the project root
cd shop-management

# 2. Activate Python Virtual Environment
# On Windows PowerShell:
.\env\Scripts\Activate.ps1

# On Windows Command Prompt (CMD):
.\env\Scripts\activate.bat
```

8.2 Database Migration & Server Execution

```
# 3. Apply all database migrations to Neon PostgreSQL
python manage.py migrate

# 4. Start the local development server
python manage.py runserver
```

8.3 System Verification & URL Endpoints

Page / Endpoint	URL Address	Access Credentials
Public Storefront	<code>http://127.0.0.1:8000/</code>	No login required (Open to all visitors)
Staff Login Portal	<code>http://127.0.0.1:8000/login/</code>	<code>admin@gmail.com</code> / <code>admin</code> (or Quick Fill)
Staff Dashboard	<code>http://127.0.0.1:8000/dashboard/</code>	Requires Staff / Superuser JWT Session
Django Administration	<code>http://127.0.0.1:8000/admin/</code>	<code>jamshed@gmail.com</code> or <code>admin@gmail.com</code>
REST API Base	<code>http://127.0.0.1:8000/api/</code>	DRF Browsable API / JSON Responses

9.0 Conclusion & Future Enhancements

9.1 Summary of Achievements

The **Shop Management System** successfully fulfills the requirements of a modern e-commerce platform and inventory system:

- A fully integrated cloud database running on serverless **Neon PostgreSQL**.
- A frictionless public home page with live search, responsive product cards, and Bangladeshi Taka (**৳**) pricing.
- An enterprise-grade administrative engine equipped with automatic inventory deduction and audit trails.
- A smooth demonstration experience featuring pre-configured demo credentials on the login portal.

9.2 Future Roadmap

- **Payment Gateway Integration:** Integrating local payment services such as bKash, Nagad, and SSLCommerz for automated online checkouts.
- **Customer Self-Service Orders:** Adding public cart and checkout flow with automated SMS notifications on order dispatch.
- **Analytics & Financial Reporting:** Interactive charting of daily sales, profit margins, and predictive stock replenishment alerts.
- **Mobile Application:** Developing a React Native or Flutter mobile client communicating via the existing DRF REST APIs.

Shop Management & E-Commerce System

Comprehensive Academic Project Documentation • Prepared for Faculty Review